**SIMATS ENGINEERING**

Department of Information Technology

# CSA07 – Computer Networks

## Assignment 2 – CO2

*IP Addressing, Subnetting and Routing for Efficient Network Design*

| Field | Details |
|---|---|
| Student Name | Jagadhratchagan |
| Register Number | 1912260614 |
| Department / Section | Information Technology / ____ |
| Academic Year / Semester | ____ |
| Faculty Name | ____ |
| Nature of Assignment | Individual |
| GitHub Repository | ____ |
| Date of Submission | ____ |

Common assumptions: network and broadcast addresses are reserved; /30 is used for two-host links; OSPF cost = reference bandwidth ÷ link bandwidth (minimum 1); RIP uses hop count with 16 as infinity; 99 is the unreachable value in the Python programs; 203.0.113.8/29 is taken as the clinic's provider block.

# Q1 – Spot the Invalid Addresses

| Device | Network | Broadcast | Usable range | Decision |
|---|---|---|---|---|
| Admission PC 172.16.45.200/20 | 172.16.32.0 | 172.16.47.255 | 172.16.32.1 – 172.16.47.254 | Valid host |
| Lab printer 10.0.15.255/21 | 10.0.8.0 | 10.0.15.255 | 10.0.8.1 – 10.0.15.254 | Invalid – broadcast address |
| X-ray server 192.168.4.0/23 | 192.168.4.0 | 192.168.5.255 | 192.168.4.1 – 192.168.5.254 | Invalid – network address |
| Clinic router 203.0.113.10/29 | 203.0.113.8 | 203.0.113.15 | 203.0.113.9 – .14 | Valid host |

**Working.** /20 = 255.255.240.0, block 16 in the third octet, and 45 falls in 32–47. /21 = 255.255.248.0, block 8, range 8–15, so 10.0.15.255 is the last address of the subnet. /23 = 255.255.254.0, the subnet starts at 4.0 and ends at 5.255, so .4.0 is the network ID. /29 = 255.255.255.248, blocks of 8, so .10 lies in .8–.15.

**The two traps are the printer (broadcast) and the X-ray server (network ID).** Both need a usable host address, for example 10.0.8.10 and 192.168.4.10.

## NAT / PAT with two PCs on source port 50000

| Inside host | Inside source | Translated source | Remote |
|---|---|---|---|
| PC-1 | 192.168.1.10:50000 | 203.0.113.10:40001 | 198.51.100.20:443 |
| PC-2 | 192.168.1.11:50000 | 203.0.113.10:40002 | 198.51.100.20:443 |

PAT gives each flow a different public source port, so both PCs can use local port 50000 at the same time. Replies to port 40001 or 40002 are matched in the NAT table and returned to the correct PC. The /29 has 8 addresses and 6 usable, but PAT lets all 18 devices share one public address.

## Datagram switching and virtual circuits

In datagram switching each packet carries the full destination address and is routed on its own. Packets of one file can take different paths with different queueing delays, so packet 3 may arrive before packet 2; TCP reorders them. A virtual circuit sets up a path first, and packets carry a short circuit ID and follow the same path. They normally arrive in order, but routers must keep per-connection state, there is a setup delay, and a failed node breaks the circuit.

## Alternative: public /27 instead of NAT

A /27 has 32 addresses and 30 usable (a /28 has only 14, which is too few). Eighteen devices fit with 12 spare, or 11 if the gateway is counted separately. Cost: a larger provider allocation, and no tariff is given, so a price cannot be calculated. Exposure: every device is directly reachable from the internet, so firewall rules become essential. NAT-related problems removed: inbound peer-to-peer reachability, port forwarding, and some SIP, FTP and VoIP application-layer problems.

# Q2 – Subnet a Campus That Keeps Growing

## A. Equal-size subnetting

The largest need is 100 hosts, so each equal subnet must be /25 (128 addresses, 126 usable). Four /25 subnets need 512 addresses and cannot fit in a /24, which holds only two. A /26 would give four subnets but only 62 hosts each, too few for the laboratories. Equal-size subnetting is therefore infeasible in 192.168.10.0/24 and needs the /23 parent block 192.168.10.0/23.

| # | Network | Mask | First host | Last host | Broadcast |
|---|---|---|---|---|---|
| 1 | 192.168.10.0/25 | 255.255.255.128 | 192.168.10.1 | 192.168.10.126 | 192.168.10.127 |
| 2 | 192.168.10.128/25 | 255.255.255.128 | 192.168.10.129 | 192.168.10.254 | 192.168.10.255 |
| 3 | 192.168.11.0/25 | 255.255.255.128 | 192.168.11.1 | 192.168.11.126 | 192.168.11.127 |
| 4 | 192.168.11.128/25 | 255.255.255.128 | 192.168.11.129 | 192.168.11.254 | 192.168.11.255 |

## B. VLSM (largest first)

| Dept | Hosts | Block | Network | Mask | First | Last | Broadcast |
|---|---|---|---|---|---|---|---|
| Labs | 100 | 128 | 192.168.10.0/25 | 255.255.255.128 | 10.1 | 10.126 | 10.127 |
| Staff | 50 | 64 | 192.168.10.128/26 | 255.255.255.192 | 10.129 | 10.190 | 10.191 |
| Library | 25 | 32 | 192.168.10.192/27 | 255.255.255.224 | 10.193 | 10.222 | 10.223 |
| WAN | 2 | 4 | 192.168.10.224/30 | 255.255.255.252 | 10.225 | 10.226 | 10.227 |

Addresses in the First, Last and Broadcast columns are in 192.168.x.y form (192.168.10.1 and so on). The next free address is 192.168.10.228. The 28 addresses left (.228 to .255) cannot hold a /26, which needs 64 aligned addresses. The largest free blocks are .228/30, .232/29 and .240/28. Even repacking cannot help: 128 + 64 + 32 + 4 + 64 = 292, which is more than 256.

**Smallest change:** widen the campus pool to 192.168.10.0/23 and put the hostel at 192.168.11.0/26 (hosts .11.1 – .11.62, broadcast .11.63). No existing subnet is renumbered.

## C. Route summarisation

172.16.8.0/24 to 172.16.11.0/24 have third octets 8–11 (00001000 to 00001011), which share the first six bits, so the summary is 172.16.8.0/22 (255.255.252.0), covering 172.16.8.0 – 172.16.11.255. If 172.16.12.0/24 is added and summarised as 172.16.8.0/21, the summary covers 172.16.8.0 – 172.16.15.255. That includes 13.0, 14.0 and 15.0, which the site does not own, so the router attracts traffic for them and drops it (a black hole), or loops it. The correct approach is to keep 8.0/22 and advertise 12.0/24 separately.

## D. Python VLSM program and verification

The program sorts the requirements largest first, gives each the smallest power-of-two block that holds hosts + 2, aligns it on its block boundary, prints the subnet details, and then prints the next free address and a hostel test. Source code (screenshots):

```python
import ipaddress

base = ipaddress.ip_network("192.168.10.0/24")

requirements = [
    ("Labs", 100),
    ("Staff", 50),
    ("Library", 25),
    ("WAN", 2)
]

requirements.sort(key=lambda item: item[1], reverse=True)

cursor = int(base.network_address)

for name, hosts in requirements:
    needed = hosts + 2
    size = 1

    while size < needed:
        size *= 2

    prefix = 32 - (size.bit_length() - 1)
    start = ((cursor + size - 1) // size) * size

    subnet = ipaddress.ip_network((start, prefix))
```

*Figure 1 – VLSM program, lines 1–26*

```
27
28      if subnet.network_address not in base or subnet.broadcast_address not in
    base:
29          print(name, "does not fit in", base)
30          continue
31
32      print(
33          name,
34          subnet,
35          subnet.netmask,
36          subnet.network_address + 1,
37          subnet.broadcast_address - 1,
38          subnet.broadcast_address
39      )
40
41      cursor = int(subnet.broadcast_address) + 1
42
43  print("Next free address:", ipaddress.ip_address(cursor))
44
45  hostel = ipaddress.ip_network("192.168.10.228/26", strict=False)
46
47  print("Hostel test:", hostel, "overlaps base:", hostel.subnet_of(base))
48  print("A /26 cannot fit in the remaining 28-address tail.")
```

*Figure 2 – VLSM program, lines 27–48*

Program output:

```
Labs 192.168.10.0/25 255.255.255.128 192.168.10.1 192.168.10.126 192.168.10.127
Staff 192.168.10.128/26 255.255.255.192 192.168.10.129 192.168.10.190 192.168.10.191
Library 192.168.10.192/27 255.255.255.224 192.168.10.193 192.168.10.222
192.168.10.223
WAN 192.168.10.224/30 255.255.255.252 192.168.10.225 192.168.10.226 192.168.10.227
Next free address: 192.168.10.228
Hostel test: 192.168.10.192/26 overlaps base: True
A /26 cannot fit in the remaining 28-address tail.
```

*Figure 3 – VLSM program output*

The output matches the table in section B: Labs 192.168.10.0/25, Staff 192.168.10.128/26, Library 192.168.10.192/27, WAN 192.168.10.224/30, and the next free address is 192.168.10.228. The last line confirms that a /26 cannot fit in the remaining 28-address tail.

## E. Alternative: smallest subnet first

Allocating the WAN /30 first at .0 uses .0–.3. The library /27 would start at .4, but a /27 must begin on a multiple of 32, so 192.168.10.4/27 is invalid and really means 192.168.10.0/27, which overlaps the WAN block. The staff /26 at .36 really means .0/26, and the labs /25 at .100 really means .0/25, so all of them overlap. Fixing it by aligning (WAN .0, Library .32, Staff .64, Labs .128) wastes .4–.31, which is 28 addresses. Largest-first stays aligned automatically, because each large block is a multiple of every smaller block size, so no gaps appear.

## F. Packet Tracer plan

Connect a core router to three LANs (Labs 192.168.10.0/25, Staff 192.168.10.128/26, Library 192.168.10.192/27) and the WAN /30, with the first usable address of each subnet as the gateway. Verify with show ip interface brief, show ip route and pings between one PC per LAN.

## Q3 – Which Way Will the Packet Go?

Rule used: the matching route with the longest prefix wins. The default route is used only when nothing else matches.

| Destination | Matching routes | Selected route | Next hop |
|---|---|---|---|
| 10.1.2.200 | /8, /16, /24, /25 | 10.1.2.128/25 | 192.0.2.13 |
| 10.1.2.100 | /8, /16, /24 | 10.1.2.0/24 | 192.0.2.9 |
| 10.1.9.9 | /8, /16 | 10.1.0.0/16 | 192.0.2.5 |
| 10.7.7.7 | /8 | 10.0.0.0/8 | 192.0.2.1 |
| 8.8.8.8 | default only | 0.0.0.0/0 | 198.51.100.1 |

10.1.2.100 is outside 10.1.2.128/25 because its last octet is below 128, so the /24 wins.

### Two-campus configuration with floating static backup

Main LAN 192.168.10.0/24 on R1 G0/0, city LAN 192.168.20.0/24 on R2, primary serial link 10.0.0.0/30 (R1 = DCE, clock rate 64000), and a backup Gigabit link 172.16.0.0/30.

```
! R1
interface g0/0
 ip address 192.168.10.1 255.255.255.0
 no shutdown
interface s0/0/0
 ip address 10.0.0.1 255.255.255.252
 clock rate 64000
 no shutdown
interface g0/1
 ip address 172.16.0.1 255.255.255.252
 no shutdown
ip route 192.168.20.0 255.255.255.0 10.0.0.2
ip route 192.168.20.0 255.255.255.0 172.16.0.2 200

! R2
interface g0/0
 ip address 192.168.20.1 255.255.255.0
 no shutdown
interface s0/0/0
 ip address 10.0.0.2 255.255.255.252
 no shutdown
interface g0/1
 ip address 172.16.0.2 255.255.255.252
 no shutdown
ip route 192.168.10.0 255.255.255.0 10.0.0.1
ip route 192.168.10.0 255.255.255.0 172.16.0.1 200
```

**Expected before failure:** R1 lists S 192.168.20.0/24 [1/0] via 10.0.0.2, and the AD 200 route stays hidden. A PC at 192.168.10.10 can ping 192.168.20.10.

**Expected after shutting s0/0/0:** the primary route is withdrawn and R1 installs S 192.168.20.0/24 [200/0] via 172.16.0.2. Pings recover after one or two lost packets.

**Removing the clock rate on the DCE end:** the serial interface stays up but the line protocol goes down, because there is no clocking. The connected route 10.0.0.0/30 disappears, pings over the primary link fail, and traffic moves to the backup route.

## Alternative: OSPF instead of static routes

| Measure | Static routes | OSPF |
|---|---|---|
| Config lines, 2 campuses | 2 routes (4 with floating backups) | about 6 (router ospf 1 plus 2 network statements per router) |
| Config lines, 5 campuses | N(N−1) = 20 routes (40 with backups) | about 15, growing linearly |
| Recovery on main link failure | Floating route takes over once the interface goes down; a silent or remote failure is not detected | Dead interval 40 s by default (about 4 s with hello 1 / dead 4, immediate on carrier loss), then SPF recomputes every route |

# Q4 – Watch Distance-Vector Routing Count to Infinity

## A. Converged distance tables

Link costs: A–B 2, A–C 7, B–C 3, B–D 8, C–D 1, D–E 4. The update rule is Dx(y) = min over neighbours v of [c(x,v) + Dv(y)].

| Round | A | B | C | D | E |
|---|---|---|---|---|---|
| 0 | 0 2 7 99 99 | 2 0 3 8 99 | 7 3 0 1 99 | 99 8 1 0 4 | 99 99 99 4 0 |
| 1 | 0 2 5 8 99 | 2 0 3 4 12 | 5 3 0 1 5 | 8 4 1 0 4 | 99 12 5 4 0 |
| 2 | 0 2 5 6 12 | 2 0 3 4 8 | 5 3 0 1 5 | 6 4 1 0 4 | 12 8 5 4 0 |
| 3 (final) | 0 2 5 6 10 | 2 0 3 4 8 | 5 3 0 1 5 | 6 4 1 0 4 | 10 8 5 4 0 |

The tables change for three rounds and a fourth pass confirms no change. A's best route to C is through B (2 + 3 = 5), which beats the direct link of cost 7. A reaches D through B and C at cost 6 (2 + 3 + 1), better than A–C–D (8) or A–B–D (10).

## B. After the C–D link fails

| From / To | A | B | C | D | E |
|---|---|---|---|---|---|
| A | 0 | 2 | 5 | 10 | 14 |
| B | 2 | 0 | 3 | 8 | 12 |
| C | 5 | 3 | 0 | 11 | 15 |
| D | 10 | 8 | 11 | 0 | 4 |
| E | 14 | 12 | 15 | 4 | 0 |

The network stays connected through B–D. C now reaches D via B at 3 + 8 = 11, and the tables settle after two changing rounds.

## C. Count to infinity, split horizon and poison reverse

Routers X–Y–Z in a line, cost 1 each. Y–Z fails. Y still hears X's vector saying Z is 2 away, so Y concludes it can reach Z through X at cost 3. X hears Y's 3 and sets 4, then Y sets 5, X sets 6, and so on, each exchange adding 2 while packets loop between X and Y, until the metric reaches infinity (16 in RIP).

| Stage | X to Z | Y to Z | Note |
|---|---|---|---|
| Before failure | 2 via Y | 1 direct | healthy |
| Y–Z fails | 2 (stale) | unreachable | Y has no direct route |
| Exchange 1 | 2 | 3 via X | Y believes X's stale 2 |
| Exchange 2 | 4 via Y | 3 | X believes Y's 3 |
| Further | 6, 8, … | 5, 7, … | rises until infinity |

Split horizon stops X advertising Z back to Y, because X learned Z from Y. Poison reverse goes further: X advertises Z as infinity to Y. Y immediately marks Z unreachable and the loop never forms. Loops across three or more routers still need triggered updates and hold-down.

## D. Python distance-vector program

The program starts every router with its direct link costs (99 for unreachable) and updates each router from its neighbours' vectors of the previous round until nothing changes. It then removes the C–D link and runs again. Source code (screenshots):

```
1  NODES = "ABCDE"
2
3  LINKS = {
4      ("A", "B"): 2,
5      ("A", "C"): 7,
6      ("B", "C"): 3,
7      ("B", "D"): 8,
8      ("C", "D"): 1,
9      ("D", "E"): 4
10 }
11
12 INF = 99
13
14 def run(links):
15     d = {
16         x: {
17             y: (0 if x == y else INF)
18             for y in NODES
19         }
20         for x in NODES
21     }
22
23     for (a, b), cost in links.items():
24         d[a][b] = d[b][a] = cost
25
26     rounds = 0
```

*Figure 4 – Distance-vector program, lines 1–26*

```
27
28      while True:
29          new = {x: d[x].copy() for x in NODES}
30
31          for x in NODES:
32              neighbours = [
33                  b if a == x else a
34                  for (a, b) in links
35                  if a == x or b == x
36              ]
37
38              for dest in NODES:
39                  for n in neighbours:
40                      cost = links.get(
41                          (x, n),
42                          links.get((n, x))
43                      )
44
45                      new[x][dest] = min(
46                          new[x][dest],
47                          cost + d[n][dest]
48                      )
49
50          rounds += 1
51
52          if new == d:
```

*Figure 5 – Distance-vector program, lines 27–52*

```
            break

        d = new

    print("Stable after", rounds - 1, "changing rounds")

    for x in NODES:
        print(x, [d[x][y] for y in NODES])

    return d


print("Original network")
run(LINKS)

print("After C-D failure")

failed = {
    edge: cost
    for edge, cost in LINKS.items()
    if set(edge) != {"C", "D"}
}

run(failed)
```

*Figure 6 – Distance-vector program, lines 53–76*

Program output:

```
Original network
Stable after 3 changing rounds
A [0, 2, 5, 6, 10]
B [2, 0, 3, 4, 8]
C [5, 3, 0, 1, 5]
D [6, 4, 1, 0, 4]
E [10, 8, 5, 4, 0]
After C-D failure
Stable after 2 changing rounds
A [0, 2, 5, 10, 14]
B [2, 0, 3, 8, 12]
C [5, 3, 0, 11, 15]
D [10, 8, 11, 0, 4]
E [14, 12, 15, 4, 0]
```

*Figure 7 – Distance-vector program output*

The output agrees with the tables above: three changing rounds for the original network and two after the C–D failure.

## E. Alternative: link-state routing

After C–D fails, routers C and D each flood one link-state advertisement. In this network (5 links left) each advertisement crosses each link at most twice, so about 10 messages per advertisement and at most about 20 in total. Every router then runs Dijkstra locally, so all routers have correct routes after roughly the network diameter in flooding hops (about 3), with no counting to infinity. Distance-vector sends a full vector over every link in every round: 10 vectors per round, about 30 messages over the three rounds, and many more if a count-to-infinity loop develops. Link-state converges faster and without loops, at the cost of more memory and CPU.

## F. RIP in Packet Tracer

```
router rip
 version 2
 no auto-summary
 network 10.0.0.0
end
show ip protocols
show ip route
```

RIP selects by hop count, not by the weighted costs above, so A would choose the direct A–C link (1 hop). The weighted exercise illustrates the algorithm. To observe convergence, ping across the network, shut one link, and watch the route table and RIP timers (update 30 s, invalid 180 s, hold-down 180 s, flush 240 s).

## Q5 – Why Does OSPF Treat 1 Gbps and 10 Gbps the Same?

| Link | Calculation (100 Mbps ref.) | Default cost | Cost with 100 Gbps ref. |
|---|---|---|---|
| 10 Mbps | 100 / 10 | 10 | 10,000 |
| 100 Mbps | 100 / 100 | 1 | 1,000 |
| 1 Gbps | 100 / 1,000 = 0.1 | 1 (minimum) | 100 |
| 10 Gbps | 100 / 10,000 = 0.01 | 1 (minimum) | 10 |

With the default reference bandwidth, 1 Gbps and 10 Gbps links both cost 1. OSPF sees equal paths and may use the 1 Gbps link. The fix is a single, consistent reference bandwidth on every router: auto-cost reference-bandwidth 100000 (100 Gbps), which also leaves room for future 100 Gbps links.

### Dijkstra by hand from R1

Network (cost with 100 Gbps reference): R1–R2 10, R1–R3 100, R2–R4 10, R3–R4 100, R3–R5 1000, R4–R6 10, R5–R6 100.

| Step | Settled node | Updated distances |
|---|---|---|
| 0 | R1 (0) | R2 = 10, R3 = 100 |
| 1 | R2 (10) | R4 = 20 |
| 2 | R4 (20) | R6 = 30 (R3 via R4 = 120, not better than 100) |
| 3 | R6 (30) | R5 = 130 |
| 4 | R3 (100) | R5 via R3 = 1100, not better |
| 5 | R5 (130) | done |

Shortest-path tree: R1→R2→R4→R6→R5 over the 10 Gbps links (R6 at cost 30, R5 at 130), and R1→R3 directly (100). With the default reference all links of 1 Gbps or more cost 1, so these paths would tie.

### Packet Tracer configuration

```
router ospf 1
 auto-cost reference-bandwidth 100000
 network <network> <wildcard> area 0
end
show ip ospf interface
show ip route ospf
```

Packet Tracer may not offer true 10 Gbps interfaces. In that case set the interface bandwidth with the bandwidth command to model relative speeds, as a metric demonstration.

### One area or several

For six routers, a single Area 0 is simplest. For a state-wide provider with many cities, a hierarchical design is better: Area 0 as the core and one area per region, with area border routers summarising each region. This keeps each link-state database small, limits how far topology changes spread and shortens SPF runs. The cost is extra design and troubleshooting complexity, and the backbone must stay connected.

## Alternative: EIGRP or IS-IS

Classic EIGRP metric = 256 × (10^7 / lowest bandwidth in kbps + total delay in tens of microseconds). Using 10 µs delay per Gigabit and 10 Gigabit interface: 1 Gbps link = 256 × (10 + 1) = 2,816; 10 Gbps link = 256 × (1 + 1) = 512. The two links get clearly different metrics with no reference bandwidth change, so the 10 Gbps path is chosen. IS-IS defaults to a metric of 10 on every interface, so it cannot tell 1 Gbps from 10 Gbps until metrics are set by hand. EIGRP avoids the problem out of the box, though it is Cisco-originated while OSPF is an open standard.

# Q6 – IPv6, ICMP, Multicast and SDN in a Smart City

**Shortest IPv6 form:** 2001:0db8:0000:0000:0000:ff00:0042:8329 becomes 2001:db8::ff00:42:8329 (leading zeros removed and the longest run of zero groups replaced by :: once).

**EUI-64:** MAC 00:1A:2B:3C:4D:5E. Insert FFFE in the middle to get 00:1A:2B:FF:FE:3C:4D:5E, then flip the universal/local bit (00 becomes 02). The interface identifier is 021a:2bff:fe3c:4d5e.

**Traceroute and ICMP:** traceroute sends probes with TTL 1, 2, 3 and so on. Each router decrements TTL, and the router where it reaches 0 discards the packet and returns ICMP Time Exceeded (type 11, code 0). The source address of that reply identifies the router. When a probe reaches the destination, it answers with Port Unreachable (UDP probes) or Echo Reply (ICMP probes), which ends the trace. Firewalls may block replies, which shows as timeouts.

**Multicast MAC for 239.1.2.3:** 01:00:5E followed by the low 23 bits of the address (1.2.3), giving 01:00:5E:01:02:03. The camera sends one stream to the group address. Receivers join with IGMP, and multicast routing (PIM) builds a tree, so routers copy the stream only at branch points towards each control room. The camera does not make a copy per receiver.

## OpenFlow rule evaluation

| Packet | Matching rules | Result |
|---|---|---|
| HTTP to 10.5.1.1 | Priority 200 (dst 10.5.0.0/16) and priority 100 (TCP 80) | Priority 200 wins: forwarded to port 3 |
| SSH to 10.6.1.1 | Only priority 0 | Dropped |

SDN is worth adopting in stages: central policy, traffic engineering and fast service changes suit a smart city. The risks are controller availability and security, so redundant controllers and secured control channels are needed.

## Alternative: IPv6 migration options

Native IPv6 only gives the cleanest long-term network and the address space for 2 million sensors, but every application and tool must support it. Dual-stack runs both protocols and keeps compatibility, at the cost of operating two stacks. A 6in4 tunnel adds a 20-byte IPv4 header, so a 100-byte sensor packet becomes 120 bytes, a 20% increase, plus MTU complications. Recommendation: start dual-stack, give new sensor networks native IPv6, and use tunnels only to connect isolated IPv6 islands over an IPv4-only core.

# Q7 – Find the Weak Point, Redesign and Validate

**Selected network:** the engineering-college campus from Q2.

## A. Identify the limitation

192.168.10.0/24 supports the initial VLSM plan, but after Labs /25, Staff /26, Library /27 and WAN /30 only 28 addresses remain (.228 to .255). The hostel needs 60 hosts, so a /26 (64 addresses). The Q2 program confirms the hostel does not fit, and 128 + 64 + 32 + 4 + 64 = 292 > 256 shows that no repacking helps. The cause is the size of the address pool in the addressing design, not router capacity.

## B. Design objective

Add a 60-host hostel subnet (at least 62 usable addresses) with zero renumbering of existing subnets and zero overlap.

## C. Revised network

Expand the campus allocation to 192.168.10.0/23 and keep every existing subnet unchanged. The hostel takes 192.168.11.0/26. Trade-off: the campus needs the adjacent 192.168.11.0/24 from the address authority, the aggregate route becomes a /23, and DHCP, ACLs and documentation need updating for the hostel.

| Segment | Network | Usable range | Broadcast | Gateway |
|---|---|---|---|---|
| Labs | 192.168.10.0/25 | 192.168.10.1 – .126 | 192.168.10.127 | 192.168.10.1 |
| Staff | 192.168.10.128/26 | 192.168.10.129 – .190 | 192.168.10.191 | 192.168.10.129 |
| Library | 192.168.10.192/27 | 192.168.10.193 – .222 | 192.168.10.223 | 192.168.10.193 |
| WAN | 192.168.10.224/30 | 192.168.10.225 – .226 | 192.168.10.227 | 192.168.10.225 |
| Hostel | 192.168.11.0/26 | 192.168.11.1 – .62 | 192.168.11.63 | 192.168.11.1 |

Topology: one core router with five interfaces, one per segment above (Labs, Staff, Library, WAN and Hostel), each using the gateway address in the table.

## D. Prediction

The original /24 plan will fail to allocate a non-overlapping /26 for the hostel. The /23 plan will allocate it, while the network IDs of Labs, Staff, Library and WAN stay unchanged. This can be tested with an overlap check and pings to each gateway.

## E. Implementation and validation

| Test | Original /24 | Redesigned /23 | Expected result |
|---|---|---|---|
| Allocate hostel /26 | Fails (28-address tail) | 192.168.11.0/26 available | Redesign meets capacity |
| Existing subnet IDs | unchanged | unchanged | No renumbering |
| Overlap check | hostel cannot fit | no overlap among all five | Pass |
| Ping inside each LAN | pass | pass | Local switching works |
| Ping across LANs | no hostel | hostel reaches all LANs | Routing works |

Both versions are built with the same tests, saved as separate Packet Tracer files, and compared using show ip route, show ip interface brief and ping results.

## F. Final defence

The objective is met when the /26 hostel subnet is allocated without overlap and the original subnet IDs remain unchanged. The new costs are extra address space and some administrative work, but four departments are not renumbered. The original /24 plan is still preferable for a small campus with no expected growth, because it is simpler and conserves address space. The /23 is preferable once the 60-host hostel is confirmed.